

Nepal Engineering College

Changunarayan, Bhaktapur



Project Report on

Telephone Simulation System

Submitted By

Dharmdev Chai (023-348)

Submitted to:

Assoc. Prof. Sundar Kunwar

Department of Computer Science and Engineering
Nepal Engineering College
Bhaktapur, Nepal

12th Aug 2026

1. Introduction

Telephone simulation models are fundamental tools in telecommunications engineering used to analyze traffic performance, resource allocation, and line availability. This Java Swing application implements discrete-event simulation techniques to model call processing under two distinct operational paradigms:

- **Lost Calls Model:** In this approach, calls are handled under a "lost calls cleared" strategy. If a subscriber attempts to place a call when either the origin line, target line, or shared system links are unavailable, the call cannot be processed and is immediately rejected (marked as blocked or busy). It is not retried automatically by the system.
- **Delayed Calls Model:** In this approach, calls operate under a "lost calls delayed" or queuing strategy. When line or link resources are insufficient to establish an immediate connection, incoming call requests are placed into a waiting queue. As soon as active calls finish and release their resources, queued calls are evaluated and connected sequentially based on availability.

1.1 Overview

The developed application is a Java Swing-based desktop telephone system simulator designed to model telecommunication network behaviors under distinct load conditions. Upon launch, the system allows the user to select between two core simulation engines:

- **Lost Call Model:** Rejects calls immediately when network resources are unavailable.
- **Delay Call Model:** Queues calls when lines or links are occupied until resources become free.

After selecting a model, the application renders a dynamic interface displaying real-time system metrics, including:

- Next incoming call parameters (Origin, Destination, Length, Arrival Time)
- Real-time status of all telephone lines (0 = Free, 1 = Occupied)
- Total link capacity and active links currently in use
- Queue of delayed calls awaiting connection (Delay Call Model)
- List of active calls currently in progress with scheduled end times

- System clock time and cumulative execution counters (Processed, Completed, Blocked, Busy)

1.2 Objective

The main objectives of this project are:

- **Simulate Dual Traffic Handling Models:** Implement and evaluate discrete-event simulation algorithms for both Lost Calls Cleared (rejection) and Delayed Calls Queued (buffered) telephone *systems*.
- **Manage Network Resources in Real Time:** Dynamically track and allocate finite system capacity—including 8 distinct subscriber lines and 3 shared switching links—to prevent routing conflicts and line collisions.
- **Provide Interactive Performance Monitoring:** Develop a graphical user interface (GUI) to visualize live system state transitions, queue behaviors, active connection lifecycles, and cumulative call metrics (Processed, Completed, Blocked, Busy).

2. Theoretical Background

2.1 Telephone System Simulation

A telephone switching network establishes point-to-point connections between subscribers using shared physical resources. Establishing a successful connection requires three simultaneous criteria:

- Calling (origin) line is free (Status = 0).
- Receiving (destination) line is free (Status = 0).
- At least one communication link is available ($\text{linksInUse} < \text{maxLinks}$).

If any of these resources are unavailable, the system handles the call using either immediate rejection (Lost Calls Model) or queue buffering (Delayed Calls Model).

2.2 Discrete-Event Simulation

The application uses discrete-event simulation (DES) to update system states at specific clock ticks rather than tracking continuous time. Key lifecycle events managed by the engine include:

- **Call Arrival:** A new call request enters the system.
- **Call Connection:** Lines and links are successfully allocated.
- **Call Completion:** Active call duration expires, releasing lines and links.

- **Call Rejection:** Unconnectable call is dropped and logged as busy/blocked.
- **Call Queueing / Dequeueing:** Unconnectable call is buffered, then connected once resources free up.

System Resources

- **Subscriber Lines:** 8 lines (indexed 1 to 8) tracked via an integer array where 0 represents free and 1 represents occupied. A connected call occupies both its source line and destination line simultaneously.
- **Switching Links:** Maximum capacity fixed at 3 concurrent links (maxLinks = 3). Every active call occupies exactly 1 link.

Table 2.1 Resource in the system

Resource	Total Capacity	Occupancy per Call
Subscriber Lines	8 lines (1–8)	2 lines (Origin + Destination)
Switching Links	3 links	1 link

2.3 Lost-Call System

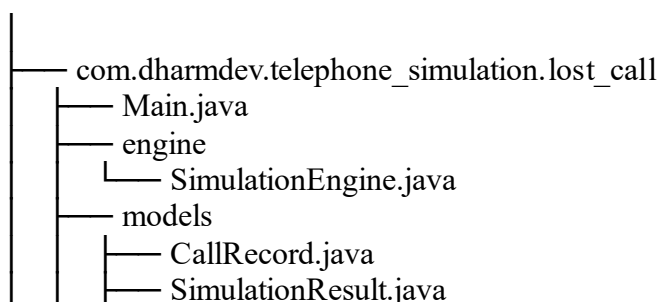
Under the Lost Calls Cleared approach, if an incoming call encounters an occupied line or exhausted link capacity, it is immediately rejected. The system increments the busy or blocked performance counter, triggers a UI dialog notification, and discards the call without placing it in a waiting list.

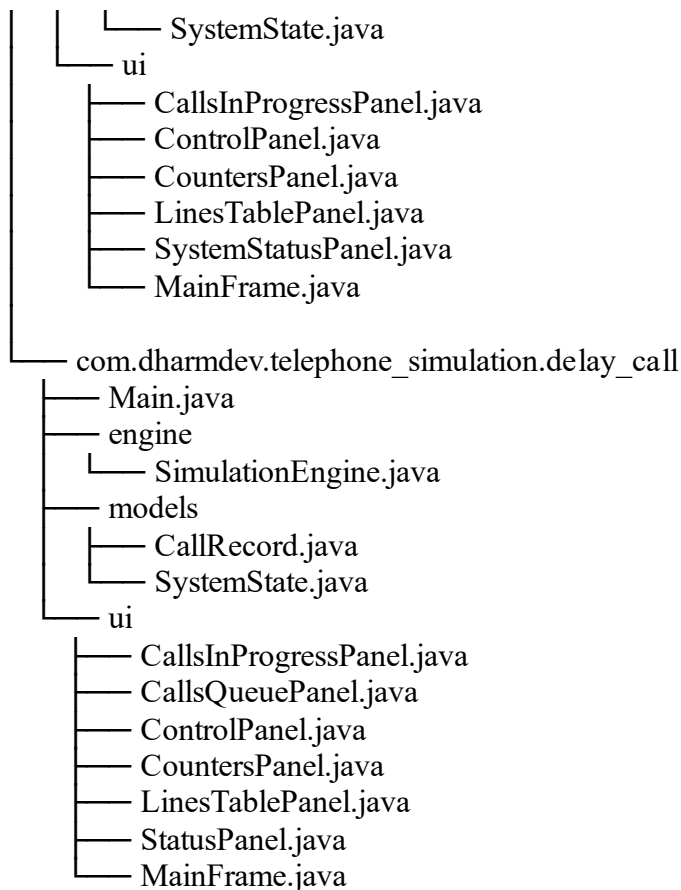
2.4 Delayed-Call System

Under the Delayed Calls Queued approach, an unconnectable call is placed into a waiting list (callsQueue). On every clock cycle tick, active calls ending their duration release their lines and links, after which the system checks processQueue(). If resources match the waiting call's requirements, it is removed from the queue and promoted to an active connection.

3. System Architecture Diagram

Telephone Simulation System





4. Implementation & Program Architecture

The application is built using standard Model-View-Controller (MVC) principles to cleanly separate simulation business logic from the Swing user interface:

- **Model Layer (models):** SystemState stores system variables including line array (int[] lines), link capacity (maxLinks = 3), current system clock, call counters, and dynamic lists (callsInProgress, callsQueue). Data models remain UI-agnostic.
- **Engine Layer (engine):** SimulationEngine manages discrete-event state updates. It utilizes a Swing Timer running on a 1000ms delay to advance the clock, evaluate call completions, release resources, and process waiting queues. UI components receive updates via a Runnable callback mechanism (onUpdateCallback).
- **User Interface Layer (ui):** Visual components inherit from JPanel and use JTable or JLabel components for real-time data display. Interface refreshes are driven by refreshUI() on the Event Dispatch Thread.

Program Execution Steps:

- **Initialize System:** Instantiate SystemState with initial line status, active links, and preset incoming call values.
- **Start Engine:** Initialize SimulationEngine passing the state reference, attach UI refresh callback, and start the background execution timer.
- **Handle Event Triggers:** On user step action or timer tick, run state validation, allocate/free resources, update counters, and refresh Swing tables.

5. Simulation Engine Logic Comparison

Action Logic	Lost Call Engine (lost_call)	Delay Call Engine (delay_call)
Call Validation	Checks if lines[from] == 1, lines[to] == 1, or linksInUse >= maxLinks.	Checks if lines[from] == 0, lines[to] == 0, and linksInUse < maxLinks.
Resource Unavailability	Increments busy or blocked counters, generates next call, and returns failure.	Adds call record to callsQueue, generates next call without dropping.
Resource Availability	Sets lines to 1, increments linksInUse, computes endTime = clock + length, adds to callsInProgress.	Sets lines to 1, increments linksInUse, computes endTime = clock + length, adds to callsInProgress.
Clock Tick Event	Frees lines/links for expired calls (clock >= endTime) and increments completed counter	Frees lines/links for expired calls, increments completed, and executes processQueue().
Queue Execution	Not applicable (no queuing mechanism).	Scans callsQueue from index 0; if target lines and links are free, dequeues call and activates it.

6. System Outputs & UI Panels

The program output is visually structured across specialized panels within MainFrame:

- **LINES Panel:** Displays an 8-row JTable rendering line index (1 to 8) and current status (0 for free, 1 for busy).

- **NEXTCALL & SYSTEM Panel:** Displays current CLOCK value, active/max LINKS capacity, and parameters for the upcoming call (Source, Target, Length, Arrival Time).
- **CALLS IN PROGRESS Panel:** Displays a table detailing active connections (FROM, TO) and scheduled completion time (END).
- **DELAYED CALLS Panel (Delay Model Only):** Renders queued calls (FROM, TO, LENGTH) awaiting line or link liberation.
- **CALL COUNTERS Panel:** A summary table rendering real-time performance tallies: PROCESSED, COMPLETED, BLOCKED, and BUSY.
- **User Notifications:** In the Lost Call system, attempt failures generate an interactive JOptionPane warning dialog stating: *"Call Failed (Busy or Blocked)! From Line: X To Line: Y"*.

7. Source Code:

7.1 Loss Call System:

SimulationEngine.java:

```
package com.dharmdev.telephone_simulation.lost_call.engine;
import com.dharmdev.telephone_simulation.lost_call.models.CallRecord;
import com.dharmdev.telephone_simulation.lost_call.models.SimulationResult;
import com.dharmdev.telephone_simulation.lost_call.models.SystemState;
import javax.swing.*;
import java.util.Iterator;
public class SimulationEngine {
    private SystemState state;
    private Timer timer;
    private Runnable onUpdateCallback;

    public SimulationEngine(SystemState state) {
        this.state = state;
    }

    public void setOnUpdateCallback(Runnable callback) {
        this.onUpdateCallback = callback;
    }

    public void startSimulation() {
        timer = new Timer(1000, e -> {
            state.incrementClock();
            checkCompletedCalls();
            if (onUpdateCallback != null) {
                onUpdateCallback.run();
            }
        })
    }
}
```

```

    });
    timer.start();
}
private void checkCompletedCalls() {
    Iterator<CallRecord> iterator = state.getCallsInProgress().iterator();
    while (iterator.hasNext()) {
        CallRecord call = iterator.next();
        if (state.getClock() >= call.getEndTime()) {
            // Free up lines and links
            state.getLines()[call.getFromLine()] = 0;
            state.getLines()[call.getToLine()] = 0;
            state.setLinksInUse(state.getLinksInUse() - 1);
            iterator.remove();
            state.incrementCompleted();
        }
    }
}

public SimulationResult processNextCall() {
    state.incrementProcessed();
    int from = state.getNextFrom();
    int to = state.getNextTo();
    int[] lines = state.getLines()
    if (lines[from] == 1 || lines[to] == 1) {
        state.incrementBusy();
        state.generateNewCall();
        return new SimulationResult(false, from, to);
    }
    if (state.getLinksInUse() >= state.getMaxLinks()) {
        state.incrementBlocked();
        state.generateNewCall();
        return new SimulationResult(false, from, to);
    }
    lines[from] = 1;
    lines[to] = 1;
    state.setLinksInUse(state.getLinksInUse() + 1);
    int endTime = state.getClock() + state.getNextLength();
    state.getCallsInProgress().add(new CallRecord(from, to, endTime));
    state.generateNewCall();
    return new SimulationResult(true, from, to);
}
}

```

CallRecord.java:

```

package com.dharmdev.telephone_simulation.lost_call.models;
public class CallRecord {
    private int fromLine;
    private int toLine;
    private int endTime;
    public CallRecord(int fromLine, int toLine, int endTime) {
        this.fromLine = fromLine;
    }
}

```

```

        this.toLine = toLine;
        this.endTime = endTime;
    }
    public int getFromLine() { return fromLine; }
    public int getToLine() { return toLine; }
    public int getEndTime() { return endTime; }
}
SimulationResult.Java:
package com.dharmdev.telephone_simulation.lost_call.models;
public class SimulationResult {
    private boolean success;
    private int fromLine;
    private int toLine;
    public SimulationResult(boolean success, int fromLine, int toLine) {
        this.success = success;
        this.fromLine = fromLine;
        this.toLine = toLine;
    }
    public boolean isSuccess() { return success; }
    public int getFromLine() { return fromLine; }
    public int getToLine() { return toLine; }
}

```

SystemState.Java:

```

package com.dharmdev.telephone_simulation.lost_call.models;
import java.util.ArrayList;
import java.util.List;
import java.util.Random;
public class SystemState {
    private int[] lines = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0}; // 8 lines (1-indexed)
    private int maxLinks = 3;
    private int linksInUse = 0;
    private int clock = 1000;
    private int nextFrom = 1;
    private int nextTo = 4;
    private int nextLength = 15;
    private int arrivalTime = 1005;
    private List<CallRecord> callsInProgress = new ArrayList<>();
    private int processed = 0;
    private int completed = 0;
    private int blocked = 0;
    private int busy = 0;
    private Random random = new Random();

    public void generateNewCall() {
        nextFrom = random.nextInt(1, 8);
        nextTo = random.nextInt(1, 8);
        while (nextTo == nextFrom) {
            nextTo = random.nextInt(1, 8);
        }
    }
}

```

```

        nextLength = 10 + random.nextInt(1, 20);
        arrivalTime = clock + random.nextInt(1, 5);
    }

    public void incrementClock() {
        clock++;
    }

    public int[] getLines() { return lines; }
    public int getMaxLinks() { return maxLinks; }
    public int getLinksInUse() { return linksInUse; }
    public void setLinksInUse(int linksInUse) { this.linksInUse = linksInUse; }
    public int getClock() { return clock; }
    public int getNextFrom() { return nextFrom; }
    public int getNextTo() { return nextTo; }
    public int getNextLength() { return nextLength; }
    public int getArrivalTime() { return arrivalTime; }
    public List<CallRecord> getCallsInProgress() { return callsInProgress; }

    public int getProcessed() { return processed; }
    public void incrementProcessed() { this.processed++; }
    public int getCompleted() { return completed; }
    public void incrementCompleted() { this.completed++; }
    public int getBlocked() { return blocked; }
    public void incrementBlocked() { this.blocked++; }
    public int getBusy() { return busy; }
    public void incrementBusy() { this.busy++; }
}

```

CallInProgress.java:

```

package com.dharmdev.telephone_simulation.lost_call.ui;
import com.dharmdev.telephone_simulation.lost_call.models.CallRecord;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.List;

public class CallsInProgressPanel extends JPanel {
    private JTable table;
    private DefaultTableModel model;

    public CallsInProgressPanel() {
        setLayout(new BorderLayout());
        setBorder(BorderFactory.createTitledBorder("CALLS IN PROGRESS"));

        String[] columns = {"FROM", "TO", "END"};
        model = new DefaultTableModel(columns, 0);
        table = new JTable(model);
        add(new JScrollPane(table), BorderLayout.CENTER);
    }

    public void updateCalls(List<CallRecord> calls) {

```

```

        model.setRowCount(0);
        for (CallRecord call : calls) {
            model.addRow(new Object[]{call.getFromLine(), call.getToLine(),
call.getEndTime()});
        }
    }
}

```

ControlPanel.java:

```

package com.dharmdev.telephone_simulation.lost_call.ui;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionListener;

public class ControlPanel extends JPanel {
    private JButton processCallButton;

    public ControlPanel(ActionListener processListener) {
        setLayout(new FlowLayout(FlowLayout.CENTER));
        processCallButton = new JButton("Simulate Next Call (Step)");
        processCallButton.addActionListener(processListener);
        add(processCallButton);
    }
}

```

CounterPanel.Java:

```

package com.dharmdev.telephone_simulation.lost_call.ui;
import com.dharmdev.telephone_simulation.lost_call.models.SystemState;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;

public class CountersPanel extends JPanel {
    private JTable table;
    private DefaultTableModel model;

    public CountersPanel() {
        setLayout(new BorderLayout());
        setBorder(BorderFactory.createTitledBorder("CALL COUNTERS"));
        String[] columns = {"PROCESSED", "COMPLETED", "BLOCKED",
"BUSY"};
        model = new DefaultTableModel(columns, 0);
        table = new JTable(model);
        table.setPreferredScrollableViewportSize(new Dimension(500, 45));
        add(new JScrollPane(table), BorderLayout.CENTER);
    }

    public void updateCounters(SystemState state) {
        model.setRowCount(0);
        model.addRow(new Object[]{
            state.getProcessed(),

```

```

        state.getCompleted(),
        state.getBlocked(),
        state.getBusy()
    });
}
}

```

LinesTablePanel.java:

```

package com.dharmdev.telephone_simulation.lost_call.ui;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
public class LinesTablePanel extends JPanel {
    private JTable table;
    private DefaultTableModel model;
    public LinesTablePanel() {
        setLayout(new BorderLayout());
        setBorder(BorderFactory.createTitledBorder("LINES"));
        String[] columns = {"Line", "Status"};
        model = new DefaultTableModel(columns, 0);
        table = new JTable(model);
        add(new JScrollPane(table), BorderLayout.CENTER);
    }
    public void updateData(int[] lines) {
        model.setRowCount(0);
        for (int i = 1; i < lines.length; i++) {
            model.addRow(new Object[]{i, lines[i]});
        }
    }
}

```

MainFrame.java:

```

package com.dharmdev.telephone_simulation.lost_call.ui;
import com.dharmdev.telephone_simulation.lost_call.engine.SimulationEngine;
import com.dharmdev.telephone_simulation.lost_call.models.SimulationResult;
import com.dharmdev.telephone_simulation.lost_call.models.SystemState;
import javax.swing.*;
import java.awt.*;
public class MainFrame extends JFrame {
    private SystemState systemState;
    private SimulationEngine engine;
    private LinesTablePanel linesPanel;
    private SystemStatusPanel statusPanel;
    private CallsInProgressPanel callsPanel;
    private CountersPanel countersPanel;
    private ControlPanel controlPanel;
    public MainFrame() {
        setTitle("Lost Call Telephone Simulation System - Figure 8-2 Model");
        setSize(950, 600);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
    }
}

```

```

systemState = new SystemState();
engine = new SimulationEngine(systemState);

linesPanel = new LinesTablePanel();
statusPanel = new SystemStatusPanel();
callsPanel = new CallsInProgressPanel();
countersPanel = new CountersPanel();
controlPanel = new ControlPanel(e -> {
    SimulationResult result = engine.processNextCall();
    refreshUI();

    if (!result.isSuccess()) {
        JOptionPane.showMessageDialog(this,
            String.format("Call Failed (Busy or Blocked)!\nFrom Line: %d\nTo Line:
%d",
                result.getFromLine(),
                result.getToLine()),
            "Simulation Notice",
            JOptionPane.WARNING_MESSAGE);
    }
});
JPanel centerGrid = new JPanel(new GridLayout(1, 2, 10, 10));
centerGrid.add(statusPanel);
centerGrid.add(callsPanel);
JPanel mainContentPanel = new JPanel(new BorderLayout(10, 10));
mainContentPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));

linesPanel.setPreferredSize(new Dimension(280, 0));
mainContentPanel.add(linesPanel, BorderLayout.WEST);
mainContentPanel.add(centerGrid, BorderLayout.CENTER);
JPanel southPanel = new JPanel(new BorderLayout(5, 5));
southPanel.setBorder(BorderFactory.createEmptyBorder(0, 10, 10, 10));
southPanel.add(countersPanel, BorderLayout.CENTER);
southPanel.add(controlPanel, BorderLayout.SOUTH);
setLayout(new BorderLayout());
add(mainContentPanel, BorderLayout.CENTER);
add(southPanel, BorderLayout.SOUTH);
engine.setOnUpdateCallback(this::refreshUI);
engine.startSimulation();
refreshUI();
}

public void refreshUI() {
    linesPanel.updateData(systemState.getLines());
    statusPanel.updateState(systemState);
    callsPanel.updateCalls(systemState.getCallsInProgress());
    countersPanel.updateCounters(systemState);
}
}

```

SystemStatusPanel.java

```
package com.dharmdev.telephone_simulation.lost_call.ui;
import com.dharmdev.telephone_simulation.lost_call.models.SystemState;
import javax.swing.*;
import java.awt.*;

public class SystemStatusPanel extends JPanel {
    private JLabel nextCallLabel;
    private JLabel linksLabel;
    private JLabel clockLabel;
    public SystemStatusPanel() {
        setLayout(new GridLayout(3, 1, 8, 8));
        setBorder(BorderFactory.createTitledBorder("NEXT CALL & SYSTEM"));
        setBorder(BorderFactory.createCompoundBorder(
            getBorder(),
            BorderFactory.createEmptyBorder(5, 5, 5, 5)
        ));
        nextCallLabel = new JLabel();
        linksLabel = new JLabel();
        clockLabel = new JLabel();
        add(nextCallLabel);
        add(linksLabel);
        add(clockLabel);
    }
    public void updateState(SystemState state) {
        nextCallLabel.setText(String.format("<html><b>NEXT CALL:</b><br>From: %d | To: %d<br>Length: %d | Arrival: %d</html>",
            state.getNextFrom(), state.getNextTo(), state.getNextLength(),
            state.getArrivalTime()));

        linksLabel.setText(String.format("<html><b>LINKS:</b><br>Max No: %d | In Use: %d</html>",
            state.getMaxLinks(), state.getLinksInUse()));

        clockLabel.setText(String.format("<html><b>CLOCK:</b> %d</html>",
            state.getClock()));
    }
}
```

Main.java:

```
package com.dharmdev.telephone_simulation.lost_call;
import com.dharmdev.telephone_simulation.lost_call.ui.MainFrame;
import javax.swing.SwingUtilities;

public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            new MainFrame().setVisible(true);
        });
    }
}
```


Output:

Lost Call Telephone Simulation System - Figure 8-2 Model

LINES	
Line	Status
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	0

NEXT CALL & SYSTEM

NEXT CALL:
From: 1 | To: 4
Length: 15 | Arrival: 1005

LINKS:
Max No: 3 | In Use: 0

CLOCK: 1006

CALLS IN PROGRESS			
FROM	TO	END	

CALL COUNTERS			
PROCESSED	COMPLETED	BLOCKED	BUSY
0	0	0	0

Simulate Next Call (Step)

Figure 7-1 Idle State

Lost Call Telephone Simulation System - Figure 8-2 Model

LINES	
Line	Status
1	1
2	0
3	0
4	1
5	0
6	0
7	0
8	0

NEXT CALL & SYSTEM

NEXT CALL:
From: 4 | To: 2
Length: 22 | Arrival: 1026

LINKS:
Max No: 3 | In Use: 1

CLOCK: 1030

CALLS IN PROGRESS			
FROM	TO	END	
1	4	1037	

CALL COUNTERS			
PROCESSED	COMPLETED	BLOCKED	BUSY
1	0	0	0

Simulate Next Call (Step)

Figure 7-2 State 2

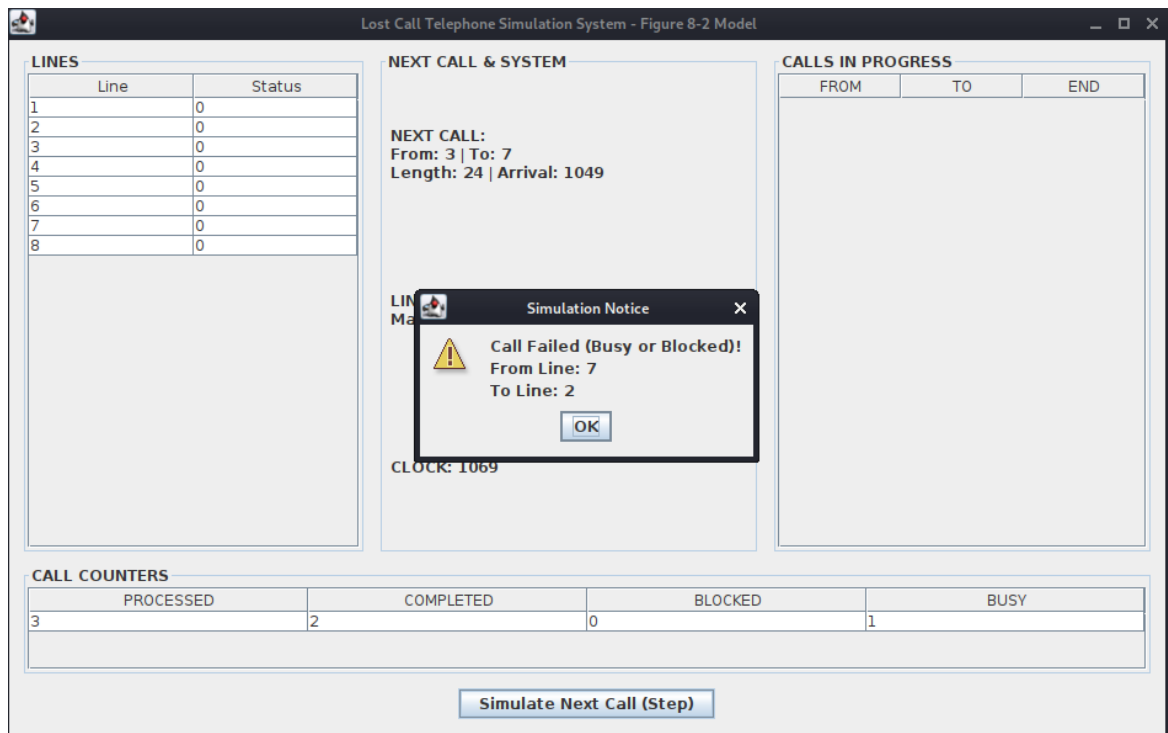


Figure 7-3 State 3

7.2 Delay Call System:

SimulationEngine.java:

```
package com.dharmdev.telephone_simulation.delay_call.engine;
import com.dharmdev.telephone_simulation.delay_call.models.CallRecord;
import com.dharmdev.telephone_simulation.delay_call.models.SystemState;
import com.dharmdev.telephone_simulation.lost_call.models.SimulationResult;
import java.util.Iterator;
import javax.swing.Timer;
public class SimulationEngine {
    private SystemState state;
    private Runnable onUpdateCallback;
    private Timer timer;
    public SimulationEngine(SystemState state) {
        this.state = state;
    }
    public void setOnUpdateCallback(Runnable callback) {
        this.onUpdateCallback = callback;
    }
    public void processNextCall() {
        state.incrementProcessed();
        int from = state.getNextFrom();
        int to = state.getNextTo();
        int length = state.getNextLength();
        int[] lines = state.getLines();
        boolean linesFree = (lines[from] == 0 && lines[to] == 0);
        boolean linksAvailable = (state.getLinksInUse() < state.getMaxLinks());
        if (linesFree && linksAvailable) {
```

```

        lines[from] = 1;
        lines[to] = 1;
        state.setLinksInUse(state.getLinksInUse() + 1);
        int endTime = state.getClock() + length;
        state.getCallsInProgress().add(new CallRecord(from, to, endTime));
    } else {
        state.getCallsQueue().add(new CallRecord(from, to, length));
    }
    state.generateNewCall();

public void tickClock() {
    state.incrementClock();

    Iterator<CallRecord> iterator = state.getCallsInProgress().iterator();
    while (iterator.hasNext()) {
        CallRecord call = iterator.next();
        if (state.getClock() >= call.getValue()) {
            state.getLines()[call.getFromLine()] = 0;
            state.getLines()[call.getToLine()] = 0;
            state.setLinksInUse(state.getLinksInUse() - 1);
            iterator.remove();
            state.incrementCompleted();
        }
    }

    processQueue()
    if (onUpdateCallback != null) {
        onUpdateCallback.run();
    }
}

private void processQueue() {
    if (!state.getCallsQueue().isEmpty() && state.getLinksInUse() <
state.getMaxLinks()) {
        CallRecord queuedCall = state.getCallsQueue().get(0);
        int from = queuedCall.getFromLine();
        int to = queuedCall.getToLine();
        if (state.getLines()[from] == 0 && state.getLines()[to] == 0) {
            state.getCallsQueue().remove(0);
            state.getLines()[from] = 1;
            state.getLines()[to] = 1;
            state.setLinksInUse(state.getLinksInUse() + 1);

            int endTime = state.getClock() + queuedCall.getValue();
            state.getCallsInProgress().add(new CallRecord(from, to, endTime));
        }
    }
}

public void startSimulation() {
    timer = new Timer(1000, e -> {
        tickClock();
        if (onUpdateCallback != null) {

```

```

        onUpdateCallback.run();
    }
});
timer.start();
}
}

```

CallRecord.java:

```

package com.dharmdev.telephone_simulation.delay_call.models;
public class CallRecord {
    private int fromLine;
    private int toLine;
    private int value;
    public CallRecord(int fromLine, int toLine, int value) {
        this.fromLine = fromLine;
        this.toLine = toLine;
        this.value = value;
    }
    public int getFromLine() { return fromLine; }
    public int getToLine() { return toLine; }
    public int getValue() { return value; }
}

```

SystemState.java:

```

package com.dharmdev.telephone_simulation.delay_call.models;
import java.util.ArrayList;
import java.util.List;
import java.util.Random;
public class SystemState {
    private int[] lines = {0, 0, 0, 0, 1, 0, 0, 1, 0};
    private int maxLinks = 3;
    private int linksInUse = 1;
    private int clock = 1057;
    private int nextFrom = 3;
    private int nextTo = 6;
    private int nextLength = 98;
    private int arrivalTime = 1063;
    private List<CallRecord> callsInProgress = new ArrayList<>();
    private List<CallRecord> callsQueue = new ArrayList<>();
    private int processed = 132;
    private int completed = 99;
    private int blocked = 5;
    private int busy = 29;
    private Random random = new Random();
    public SystemState() {
        callsInProgress.add(new CallRecord(4, 7, 1075));
        callsQueue.add(new CallRecord(1, 7, 20));
    }

    public void incrementClock() { clock++; }
}

```

```

public void generateNewCall() {
    nextFrom = 1 + random.nextInt(8);
    nextTo = 1 + random.nextInt(8);
    while (nextTo == nextFrom) {
        nextTo = 1 + random.nextInt(8);
    }
    nextLength = 10 + random.nextInt(50);
    arrivalTime = clock + 5 + random.nextInt(10);

    public int[] getLines() { return lines; }
    public int getMaxLinks() { return maxLinks; }
    public int getLinksInUse() { return linksInUse; }
    public void setLinksInUse(int linksInUse) { this.linksInUse = linksInUse; }
    public int getClock() { return clock; }
    public int getNextFrom() { return nextFrom; }
    public int getNextTo() { return nextTo; }
    public int getNextLength() { return nextLength; }
    public int getArrivalTime() { return arrivalTime; }
    public List<CallRecord> getCallsInProgress() { return callsInProgress; }
    public List<CallRecord> getCallsQueue() { return callsQueue; }
    public int getProcessed() { return processed; }
    public void incrementProcessed() { this.processed++; }
    public int getCompleted() { return completed; }
    public void incrementCompleted() { this.completed++; }
    public int getBlocked() { return blocked; }
    public int getBusy() { return busy; }
}

CallsInProgressPanel.java:
package com.dharmdev.telephone_simulation.delay_call.ui;
import com.dharmdev.telephone_simulation.delay_call.models.CallRecord;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.List;
public class CallsInProgressPanel extends JPanel {
    private JTable table;
    private DefaultTableModel model;
    public CallsInProgressPanel() {
        setLayout(new BorderLayout());
        setBorder(BorderFactory.createTitledBorder("CALLS IN PROGRESS"));
        model = new DefaultTableModel(new String[]{"FROM", "TO", "END"}, 0);
        table = new JTable(model);
        add(new JScrollPane(table), BorderLayout.CENTER);
    }
    public void updateData(List<CallRecord> calls) {
        model.setRowCount(0);
        for (CallRecord call : calls) {
            model.addRow(new Object[]{call.getFromLine(), call.getToLine(),
call.getValue()});

```

```

    }
}
}

```

CallsQueuePanel.java

```

package com.dharmdev.telephone_simulation.delay_call.ui;
import com.dharmdev.telephone_simulation.delay_call.models.CallRecord;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.List;
public class CallsQueuePanel extends JPanel {
    private JTable table;
    private DefaultTableModel model;

    public CallsQueuePanel() {
        setLayout(new BorderLayout());
        setBorder(BorderFactory.createTitledBorder("DELAYED CALLS"));
        model = new DefaultTableModel(new String[]{"FROM", "TO", "LENGTH"},
0);
        table = new JTable(model);
        add(new JScrollPane(table), BorderLayout.CENTER);
    }
    public void updateData(List<CallRecord> calls) {
        model.setRowCount(0);
        for (CallRecord call : calls) {
            model.addRow(new Object[]{call.getFromLine(), call.getToLine(),
call.getValue()});
        }
    }
}

```

ControlPanel.java

```

package com.dharmdev.telephone_simulation.delay_call.ui
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionListener;

public class ControlPanel extends JPanel {
    private JButton processCallButton;

    public ControlPanel(ActionListener processListener) {
        setLayout(new FlowLayout(FlowLayout.CENTER));
        processCallButton = new JButton("Simulate Next Call (Step)");
        processCallButton.addActionListener(processListener);
        add(processCallButton);
    }
}

```

CountersPanel.java

```

package com.dharmdev.telephone_simulation.delay_call.ui;
import com.dharmdev.telephone_simulation.delay_call.models.SystemState;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;

public class CountersPanel extends JPanel {
    private JTable table;
    private DefaultTableModel model;

    public CountersPanel() {
        setLayout(new BorderLayout());
        setBorder(BorderFactory.createTitledBorder("CALL COUNTERS"));
        model = new DefaultTableModel(new String[]{"PROCESSED",
"COMPLETED", "BLOCKED", "BUSY"}, 0);
        table = new JTable(model);
        table.setPreferredScrollableViewportSize(new Dimension(500, 45));
        add(new JScrollPane(table), BorderLayout.CENTER);
    }

    public void updateData(SystemState state) {
        model.setRowCount(0);
        model.addRow(new Object[]{state.getProcessed(), state.getCompleted(),
state.getBlocked(), state.getBusy()});
    }
}

```

DelayedCallsPanel.java

```

package com.dharmdev.telephone_simulation.delay_call.ui;
import com.dharmdev.telephone_simulation.delay_call.models.CallRecord;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.List;

public class DelayedCallsPanel extends JPanel {
    private JTable table;
    private DefaultTableModel model;

    public DelayedCallsPanel() {
        setLayout(new BorderLayout());
        setBorder(BorderFactory.createTitledBorder("DELAYED CALLS"));

        String[] columns = {"FROM", "TO", "LENGTH"};
        model = new DefaultTableModel(columns, 0);
        table = new JTable(model);

        add(new JScrollPane(table), BorderLayout.CENTER);
    }

    public void updateDelayedCalls(List<CallRecord> delayedCalls) {

```

```

        model.setRowCount(0);
        for (CallRecord call : delayedCalls) {
            model.addRow(new Object[]{call.getFromLine(), call.getToLine(),
call.getValue()});
        }
    }
}

```

LinesTablePanel.java

```

package com.dharmdev.telephone_simulation.delay_call.ui;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
public class LinesTablePanel extends JPanel {
    private JTable table;
    private DefaultTableModel model;

    public LinesTablePanel() {
        setLayout(new BorderLayout());
        setBorder(BorderFactory.createTitledBorder("LINES"));
        model = new DefaultTableModel(new String[]{"Line", "Status"}, 0);
        table = new JTable(model);
        add(new JScrollPane(table), BorderLayout.CENTER);
    }

    public void updateData(int[] lines) {
        model.setRowCount(0);
        for (int i = 1; i < lines.length; i++) {
            model.addRow(new Object[]{i, lines[i]});
        }
    }
}

```

MainFrame.java

```

package com.dharmdev.telephone_simulation.delay_call.ui;
import com.dharmdev.telephone_simulation.delay_call.engine.SimulationEngine;
import com.dharmdev.telephone_simulation.delay_call.models.SystemState;
import javax.swing.*;
import java.awt.*;
public class MainFrame extends JFrame {
    private SystemState state;
    private SimulationEngine engine;

    private LinesTablePanel linesPanel;
    private StatusPanel statusPanel;
    private CallsQueuePanel queuePanel;
    private CallsInProgressPanel progressPanel;
    private CountersPanel countersPanel;
    private ControlPanel controlPanel;

    private Timer autoTimer;
}

```

```

private JButton stopAutoBtn;

public MainFrame() {
    setTitle("Simulation System - Auto Simulation");
    setSize(1100, 650);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);

    state = new SystemState();
    engine = new SimulationEngine(state);

    linesPanel = new LinesTablePanel();
    statusPanel = new StatusPanel();
    queuePanel = new CallsQueuePanel();
    progressPanel = new CallsInProgressPanel();
    countersPanel = new CountersPanel();

    JPanel centerGrid = new JPanel(new GridLayout(1, 3, 8, 8));
    centerGrid.add(statusPanel);
    centerGrid.add(queuePanel);
    centerGrid.add(progressPanel);

    JPanel mainContent = new JPanel(new BorderLayout(10, 10));
    mainContent.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
    linesPanel.setPreferredSize(new Dimension(240, 0));
    mainContent.add(linesPanel, BorderLayout.WEST);
    mainContent.add(centerGrid, BorderLayout.CENTER);

    controlPanel = new ControlPanel(e -> {
        engine.processNextCall(); refreshUI();
    });

    JPanel southPanel = new JPanel(new BorderLayout(5, 5));
    southPanel.setBorder(BorderFactory.createEmptyBorder(0, 10, 10, 10));
    southPanel.add(countersPanel, BorderLayout.CENTER);
    southPanel.add(controlPanel, BorderLayout.SOUTH);

    setLayout(new BorderLayout());
    add(mainContent, BorderLayout.CENTER);
    add(southPanel, BorderLayout.SOUTH);

    engine.setOnUpdateCallback(this::refreshUI);
    engine.startSimulation();
    refreshUI();
}

public void refreshUI() {
    linesPanel.updateData(state.getLines());
    statusPanel.updateState(state);
    queuePanel.updateData(state.getCallsQueue());
}

```

```
        progressPanel.updateData(state.getCallsInProgress());  
        countersPanel.updateData(state);  
    }  
}
```

StatusPanel.java

```
package com.dharmdev.telephone_simulation.delay_call.ui;
import com.dharmdev.telephone_simulation.delay_call.models.SystemState;
import javax.swing.*;
import java.awt.*;

public class StatusPanel extends JPanel {
    private JLabel nextCallLabel, linksLabel, clockLabel;

    public StatusPanel() {
        setLayout(new GridLayout(3, 1, 5, 5));
        setBorder(BorderFactory.createTitledBorder("NEXT CALL & SYSTEM"));
        nextCallLabel = new JLabel();
        linksLabel = new JLabel();
        clockLabel = new JLabel();
        add(nextCallLabel);
        add(linksLabel);
        add(clockLabel);
    }

    public void updateState(SystemState state) {
        nextCallLabel.setText(String.format("<html><b>NEXT CALL:</b><br>From: %d | To: %d<br>Length: %d | Arrival: %d</html>",
            state.getNextFrom(), state.getNextTo(), state.getNextLength(),
            state.getArrivalTime()));
        linksLabel.setText(String.format("<html><b>LINKS:</b><br>Max No: %d | In Use: %d</html>",
            state.getMaxLinks(), state.getLinksInUse()));
        clockLabel.setText(String.format("<html><b>CLOCK:</b> %d</html>",
            state.getClock()));
    }
}
```

Main.Java:

```
package com.dharmdev.telephone_simulation.delay_call;
import com.dharmdev.telephone_simulation.delay_call.ui.MainFrame;
import javax.swing.SwingUtilities;

public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater() -> {
            new MainFrame().setVisible(true);
        });
    }
}
```

Output:

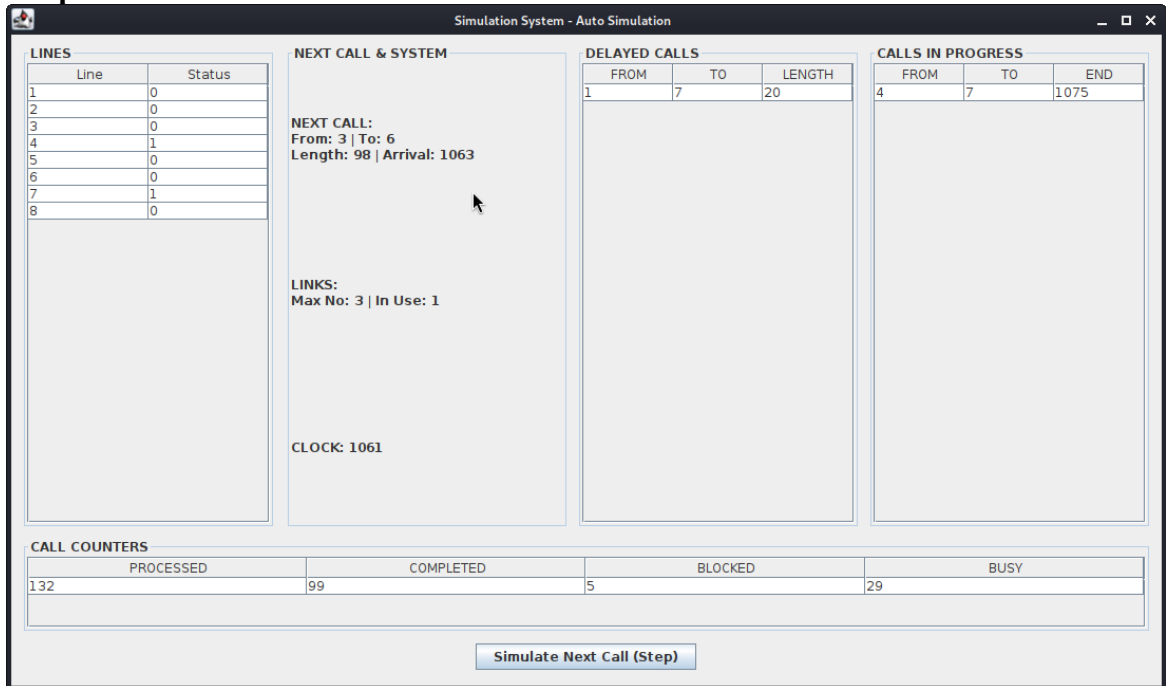


Figure 7-4 Idle State

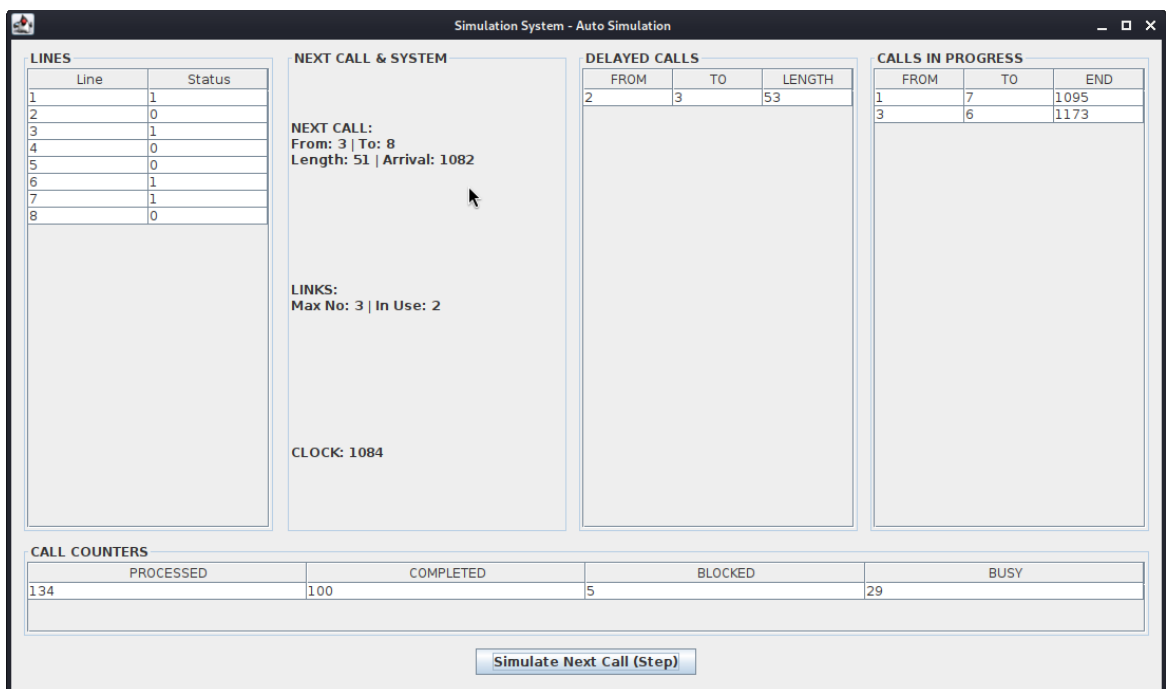


Figure 7-5 Delayed Call State

8. Conclusion

This Java Swing simulation successfully models discrete-event telephone traffic processing across two distinct operational approaches:

- **Lost Call Model:** Rejects unserved calls immediately when lines or links are occupied, prioritizing low latency over call completion.
- **Delayed Call Model:** Buffers incoming calls in a waiting queue until resources free up, maximizing call completion at the cost of waiting delay.

The application effectively demonstrates real-time resource allocation (8 subscriber lines, 3 shared links) and provides a clear interactive interface for tracking active connections, queue behavior, and system performance counters.